

Project 1 KRR

Exercise 1:

a)

I have created a KB and a Question logically entailed from KB, been inspired by the example of the last slide of Course 3, PDF.

The general problem: At one statistics office, some people wants to determine the financial status of population, where they wealth came from, and how many can party having their wealth, knowing the following rules.

My KB:

- 1.Anybody who work or is a lottery winner, has a house.
- 2.Anybody who has a house, have enough payment.
- 3.Anybody who has enough payment, can party.
- 4.Who work, isn't a lottery winner.
- 5.Andrei is a lottery winner.

Question: Andrei can party?

b)

KB in FOL:

- 1.for any x: $\text{work}(x) \vee \text{lottery}(x) \rightarrow \text{house}(x)$.
- 2.for any x: $\text{house}(x) \rightarrow \text{enoughPayment}(x)$.
- 3.for any x: $\text{enoughPayment}(x) \rightarrow \text{party}(x)$.

4. for any x : $\text{work}(x) \rightarrow \text{NON lottery}(x)$.

5. $\text{lottery}(\text{Andrei})$.

Question: $\text{party}(\text{Andrei})$.

KB in CNF:

1. $[\neg(\text{work}(x)), \text{house}(x)]$

1. $[\neg(\text{lottery}(x)), \text{house}(x)]$

2. $[\neg(\text{house}(x)), \text{enoughPayment}(x)]$

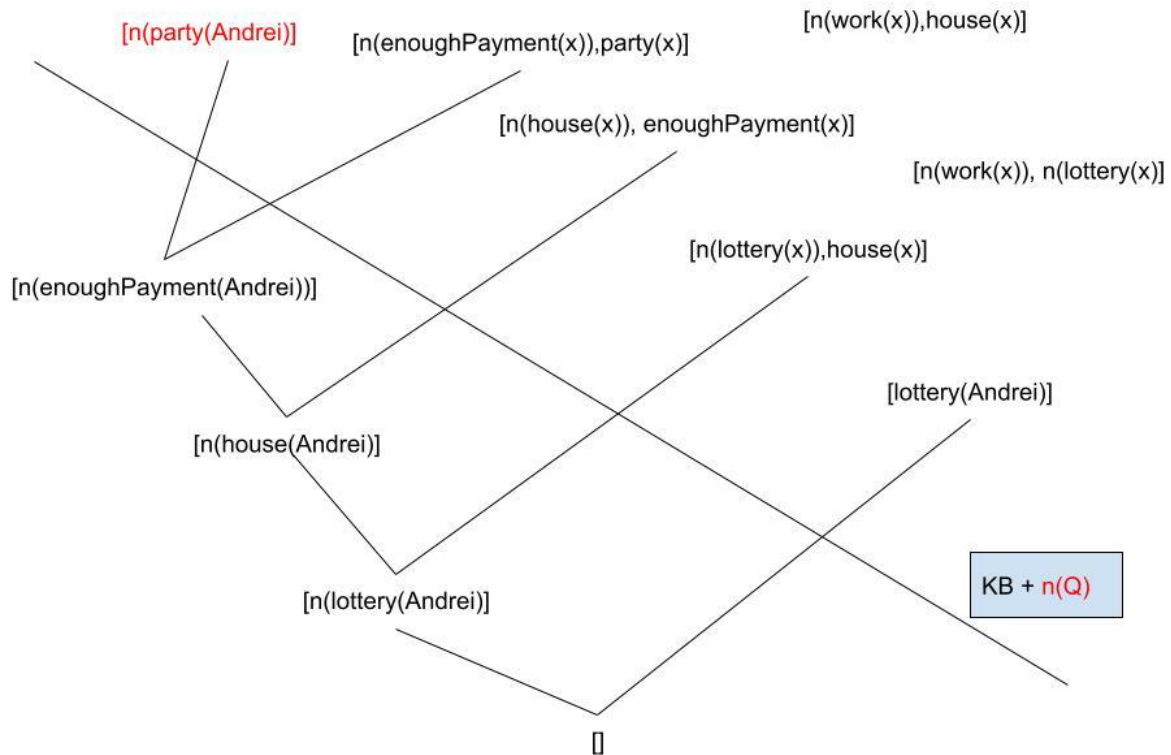
3. $[\neg(\text{enoughPayment}(x)), \text{party}(x)]$

4. $[\neg(\text{work}(x)), \neg(\text{lottery}(x))]$

5. $[\text{lottery}(\text{Andrei})]$

Question: $[\text{party}(\text{Andrei})]$.

The resolution applied manually:



c)

The KB in CNF is written in a txt in this form

[[n(work(x)), house(x)], [n(lottery(x)), house(x)], [n(house(x)), enoughPayment(x)],
[n(enoughPayment(x)), party(x)], [n(work(x)), n(lottery(x))], [lottery(Andrei)], [n(party(Andrei))]],

where [n(party(Andrei))] is the negated question. This KB will be read by SWI – prolog console, and then the resolution algorithm will be applied, which will return “SAT”, or “UNSAT”. In this case will return “UNSAT”, since the question is logically entailed from KB, in the KB we added the negated question, and because with the resolution we obtained “[]” element.

d)

Applying the resolution algorithm for following KB, we obtain:

i. $[[\neg a, b], [c, d], [\neg d, b], [\neg b], [\neg c, b], [e], [a, b, \neg f, f]] \Rightarrow \text{"UNSAT"}$.

ii. $[[\neg b, a], [\neg a, b, e], [a, \neg e], [\neg a], [e]] \Rightarrow \text{"UNSAT"}$.

iii. $[[\neg a, b], [c, f], [\neg c], [\neg f, b], [\neg c, b]] \Rightarrow \text{"SAT"}$.

iv. $[[a, b], [\neg a, \neg b], [c]] \Rightarrow \text{"SAT"}$.

CODE:

```
negate(n(A),A).
negate(A,n(A)).

merge_lists_no_duplicates(LIST1, LIST2, RESULT):-
    append(LIST1,LIST2,R), list_to_set(R, RESULT).

get_neg_val([],_,R1,R1).
get_neg_val([H|TAIL],L2,R1,L):-
    negate(A,N),
    member(N,L2),
    append(R1,[A,N],R1_NEW),
    get_neg_val(TAIL,L2,R1_NEW,L).
get_neg_val([A|TAIL],L2,R1,L):-
    get_neg_val(TAIL,L2,R1,L).

subtract_helper(KB,_,[],EXT_M,EXT_M).
subtract_helper(KB,[A,B],[[NA,NB]|N_TAIL],EXT_MEMBERS,L):-
    subtract(A,[NA],A_SUBS),
    subtract(B,[NB],B_SUBS),
```

```

merge_lists_no_duplicates(A_SUBS,B_SUBS,NEW_LIST),
sort(NEW_LIST,N_L_S),
\+member(N_L_S,KB),
append(EXT_MEMBERS,[N_L_S], EXT_MEMBERS_UP),
subtract_helper(KB,[A,B],N_TAIL,EXT_MEMBERS_UP, L).

subtract_helper(KB,[A,B],[N_HEAD|N_TAIL],EXT_MEMBERS,L):-
    subtract_helper(KB,[A,B],N_TAIL,EXT_MEMBERS,L).

get_new_members(KB,[],NEW_M,NEW_M).

get_new_members(KB,[A|COMB],NEW_M, L):-
    get_neg_val(A,[],N_VAL),
    \+ N_VAL = [],
    subtract_helper(KB,A,N_VAL,[],EXT_MEMBERS),
    append(NEW_M, EXT_MEMBERS, NEW_M_UP),
    get_new_members(KB,COMB,NEW_M_UP, L).

get_new_members(KB,[A|COMB],NEW_M, L):-
    get_new_members(KB,COMB,NEW_M, L).

combinate(_,[]):- true.
combinate([A|TAIL],[A|COMBIMATE]):-
    combine(TAIL,COMBIMATE).
combinate([_|TAIL],[A|COMBIMATE]):-
    combine(TAIL,[A|COMBIMATE]).

res(KB):-member([],KB), print("UNSAT").

res(KB):-
    findall([A,B],combinate(KB,[A,B]),COMB),
    get_new_members(KB,COMB,[],NEW_M),
    \+ NEW_M =[],
    append(KB,NEW_M, NEW_KB),
    print('new members'),
    print(NEW_M),
    print('new KB'),
    print(NEW_KB),
    res(NEW_KB).

res(KB):-print("SAT").

```

```
sort_KB([],S,S).
```

```
sort_KB([A|B],S,L):-  
    sort(A,C),  
    append(S,[C],S2),  
    sort_KB(B,S2,L).
```

```
before_res(KB,KB_S):-  
    sort_KB(KB,[],KB_S),  
    res(KB_S).
```